

Title:	A General Property Customization Mechanism
Authors:	David Hollman, dshollm@sandia.gov
	Chris Kohlhoff, chris@kohlhoff.com
	Bryce Lelbach, brycelelbach@gmail.com
	Jared Hoberock, jhoberock@nvidia.com
	Gordon Brown, gordon@codeplay.com
	Michał Dominiak, griwes@griwes.info
Other Contributors:	Lee Howes, lwh@fb.com
	Michael Garland, mgarland@nvidia.com
	Chris Mysen, mysen@google.com
	Thomas Rodgers, rodgert@twrodders.com
	Michael Wong, michael@codeplay.com
Document Number:	P1393R0
Date:	2019-01-13
Audience:	LEWG
Reply-to:	sg1-exec@googlegroups.com
Abstract:	This paper generalizes and extracts the property customization mechanism from P0443r8, as requested by LEWG in the 2018-11 San Diego meeting. This document does not introduce any significant design changes from the design previously discussed in the context of P0443; the separation herein is merely a recognition of the mechanism's general applicability and is made in anticipation of its use in other contexts.

Changelog

Revision 0

- Initial design, migrated from P0443R9.

Introduction

At the 2018-11 San Diego meeting, LEWG voted to generalize the mechanism for property-based customization that P0443R9 introduced. They requested that the customization points objects for handling properties be moved to the

namespace `std` (from namespace `std::execution`), and that a paper presenting wording for the mechanism in a manner decoupled from executors be brought to the 2019-02 Kona meeting. LEWG further requested that a separate customization point object be provided for properties that are intended to enforce the presence of a particular interface, and this paper includes that object as `require_concept` . The requested changes have been provided here. Discussion pertaining to the design of this mechanism has been omitted here, since significant background and discussion has been included in previous revisions of P0443 and meeting notes on the discussion thereof.

Proposed Wording

Add the following row to the table in `[utilities.general]`:

	Subclause	Header(s)
<code>[properties]</code>	Support for the property customization mechanism	<code><property></code>

Add the following subclause to `[utilities]` in a section which the editor shall determine:

Properties Support

General

This subclause describes components supporting an extensible customization mechanism, currently used most prominently by the execution support library `[execution]`.

Header `<property>` synopsis

```
namespace std {

    // Customization point objects:

    inline namespace unspecified {
        inline constexpr unspecified require_concept = unspecified;
        inline constexpr unspecified require = unspecified;
        inline constexpr unspecified prefer = unspecified;
        inline constexpr unspecified query = unspecified;
    }

    // Property applicability trait:
    template<class T, class P> struct is_applicable_property;

    template<class T, class Property>
        inline constexpr bool is_applicable_property_v = is_applicable_property<T, Property>::value;

    // Customization point type traits:
    template<class T, class P> struct can_require_concept;
    template<class T, class... P> struct can_require;
    template<class T, class... P> struct can_prefer;
    template<class T, class P> struct can_query;

    template<class T, class Property>
        inline constexpr bool can_require_concept_v = can_require_concept<T, Property>::value;
    template<class T, class... Properties>
        inline constexpr bool can_require_v = can_require<T, Properties...>::value;
    template<class T, class... Properties>
        inline constexpr bool can_prefer_v = can_prefer<T, Properties...>::value;
    template<class T, class Property>
        inline constexpr bool can_query_v = can_query<T, Property>::value;

} // namespace std
```

Customization point objects

`require_concept`

```
inline namespace unspecified {
    inline constexpr unspecified require_concept = unspecified;
}
```

The name `require_concept` denotes a customization point object. The expression `std::require_concept(E, P)` for some subexpressions `E` and `P` (with types `T = decay_t<decltype(E)>` and `Prop = decay_t<decltype(P)>`) is expression-equivalent to:

- If `is_applicable_property_v<T, Prop> && Prop::is_requirable_concept` is not a well-formed constant expression with value `true`, `std::require_concept(E, P)` is ill-formed.
- Otherwise, `E` if the expression `Prop::template static_query_v<T> == Prop::value()` is a well-formed constant expression with value `true`.
- Otherwise, `(E).require_concept(P)` if the expression `(E).require_concept(P)` is well-formed.

- Otherwise, `require_concept(E, P)` if the expression `require_concept(E, P)` is a valid expression with overload resolution performed in a context that does not include the declaration of the `require_concept` customization point object.
- Otherwise, `std::require_concept(E, P)` is ill-formed.

require

```
inline namespace unspecified {
    inline constexpr unspecified require = unspecified;
}
```

The name `require` denotes a customization point object. The expression `std::require(E, P0, Pn...)` for some subexpressions `E` and `P0`, and where `Pn...` represents `N` subexpressions (where `N` is 0 or more, and with types `T = decay_t<decltype(E)>` and `Prop0 = decay_t<decltype(P0)>`) is expression-equivalent to:

- If `is_applicable_property_v<T, Prop0>` && `Prop0::is_requirable` is not a well-formed constant expression with value `true`, `std::require(E, P0, Pn...)` is ill-formed.
- Otherwise, `E` if `N == 0` and the expression `Prop0::template static_query_v<T> == Prop0::value()` is a well-formed constant expression with value `true`.
- Otherwise, `(E).require(P0)` if `N == 0` and the expression `(E).require(P0)` is a valid expression.
- Otherwise, `require(E, P)` if `N == 0` and the expression `require(E, P)` is a valid expression with overload resolution performed in a context that does not include the declaration of the `require` customization point object.
- Otherwise, `std::require(std::require(E, P0), Pn...)` if `N > 0` and the expression `std::require(std::require(E, P0), Pn...)` is a valid expression.
- Otherwise, `std::require(E, P0, Pn...)` is ill-formed.

prefer

```
inline namespace unspecified {
    inline constexpr unspecified prefer = unspecified;
}
```

The name `prefer` denotes a customization point object. The expression `std::prefer(E, P0, Pn...)` for some subexpressions `E` and `P0`, and where `Pn...` represents `N` subexpressions (where `N` is 0 or more, and with types `T = decay_t<decltype(E)>` and `Prop0 = decay_t<decltype(P0)>`) is expression-equivalent to:

- If `is_applicable_property_v<T, Prop0>` && `Prop0::is_preferable` is not a well-formed constant expression with value `true`, `std::prefer(E, P0, Pn...)` is ill-formed.
- Otherwise, `E` if `N == 0` and the expression `Prop0::template static_query_v<T> == Prop0::value()` is a well-formed constant expression with value `true`.
- Otherwise, `(E).require(P0)` if `N == 0` and the expression `(E).require(P0)` is a valid expression.
- Otherwise, `require(E, P0)` if `N == 0` and the expression `require(E, P0)` is a valid expression with overload resolution performed in a context that does not include the declaration of the `require` customization point object.
- Otherwise, `std::prefer(std::prefer(E, P0), Pn...)` if `N > 0` and the expression `std::prefer(std::prefer(E, P0), Pn...)` is a valid expression.
- Otherwise, `std::prefer(E, P0, Pn...)` is ill-formed.

query

```
inline namespace unspecified {
    inline constexpr unspecified query = unspecified;
}
```

The name `query` denotes a customization point object. The expression `std::query(E, P)` for some subexpressions `E` and `P` (with types `T = decay_t<decltype(E)>` and `Prop = decay_t<decltype(P)>`) is expression-equivalent to:

- If `is_applicable_property_v<T, Prop>` is not a well-formed constant expression with value `true`, `std::query(E, P)` is ill-formed.
- Otherwise, `Prop::template static_query_v<T>` if the expression `Prop::template static_query_v<T>` is a well-formed constant expression.
- Otherwise, `(E).query(P)` if the expression `(E).query(P)` is well-formed.
- Otherwise, `query(E, P)` if the expression `query(E, P)` is a valid expression with overload resolution performed in a context that does not include the declaration of the `query` customization point object.
- Otherwise, `std::query(E, P)` is ill-formed.

Property applicability trait

```
template<class T, class Property> struct is_applicable_property;
```

This sub-clause contains a template that may be used to query the applicability of a property to a type at compile time. It may be specialized to indicate applicability of a property to a type. This template is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<code>template<class T, class P></code> <code>struct is_applicable_property</code>	The expression <code>P::template is_applicable_property_v<T></code> is a well-formed constant expression with a value of <code>true</code> .	<code>P</code> and <code>T</code> are complete types.

Customization point type traits

```
template<class T, class Property> struct can_require_concept;
template<class T, class... Properties> struct can_require;
template<class T, class... Properties> struct can_prefer;
template<class T, class Property> struct can_query;
```

This sub-clause contains templates that may be used to query the validity of the application of property customization point objects to a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<pre>template<class T, class P> struct can_require_concept</pre>	The expression <pre>std::require_concept(declval<const T>(), declval<P>())</pre> is well-formed.	<code>T</code> and <code>P</code> are complete types.
<pre>template<class T, class... P> struct can_require</pre>	The expression <pre>std::require(declval<const T>(), declval<P>()...) is</pre> well-formed.	<code>T</code> and <code>P...</code> are complete types.
<pre>template<class T, class... P> struct can_prefer</pre>	The expression <pre>std::prefer(declval<const T>(), declval<P>()...) is</pre> well-formed.	<code>T</code> and <code>P...</code> are complete types.
<pre>template<class T, class P> struct can_query</pre>	The expression <pre>std::query(declval<const T>(), declval<P>()) is well-</pre> formed.	<code>T</code> and <code>P</code> are complete types.
<pre>template<class T, class P> struct is_applicable_property</pre>	The expression <code>P::template is_applicable_property_v<T></code> is a well-formed constant expression with a value of <code>true</code>.	<code>P</code> and <code>T</code> are complete types.

The property customization mechanism

In general

When the property customization mechanism is being employed for some library facility, an object's behavior and effects on that facility in generic contexts may be determined by a set of applicable properties, and each property imposes certain requirements on that object's behavior or exposes some attribute of that object. As well as modifying the behavior of an object, properties can be applied to an object to enforce the presence of an interface, potentially resulting in an object of a new type that satisfies some concept associated with that property.

Requirements on properties

- A property type `P` shall provide a nested constant variable template named `is_applicable_property_v`, usable as `P::template is_applicable_property_v<T>` on any complete type `T`, with a value of type `bool`.
- A property type shall be either a concept-preserving property type or a concept-enforcing property type.
 - A *concept-preserving property type* `PN` shall provide:
 - A nested constant expression named `is_requirable` of type `bool`, usable as `PN::is_requirable`.
 - A nested constant expression named `is_preferable` of type `bool`, usable as `PN::is_preferable`.
 - A *concept-enforcing property type* `PC` shall provide a nested constant expression named `is_requirable_concept` of type `bool`, usable as `PC::is_requirable_concept`.
- A property type `P` may provide a nested type `polymorphic_query_result_type` that satisfies the `CopyConstructible` and `Destructible` requirements. If `P` is a concept-preserving property, and `P::is_requirable == true` or `P::is_preferable == true`, then `polymorphic_query_result_type` shall also satisfy the `DefaultConstructible` requirements when provided. [Note: When present, this type allows the property to be used with polymorphic wrappers. —end note]

- A property type `P` may provide:
 - A nested variable template `static_query_v`, usable as `P::template static_query_v<T>` for any type `T` where `is_applicable_property_v<T, P>` is `true`. [Note: This may be conditionally present. —end note]
 - A member function `value()`.
- For a property type `P` and type `T` where `is_applicable_property_v<T, P>` is `true`, if `P::value()` is a valid expression of type `V1` and `P::template static_query_v<T>` is a valid constant expression of type `V2`, then `V1` and `V2` shall meet the requirements of `EqualityComparableWith<V1, V2>` in **[concept.equalitycomparable]**.
- [Note: The `static_query_v` and `value()` members are used to determine whether invoking `require` or `require_concept` would result in an identity transformation. —end note]
- A concept-enforcing property type `P` may provide a nested template or named `polymorphic_wrapper_type`, usable with properties `Ps...` as `typename P::template polymorphic_wrapper_type<Ps...>`. The type `PW` resulting from the instantiation of this template shall:
 - be implicitly constructible from a (potentially `const`) instance of a type `T`, where:
 - `can_query_v<Pn, T>` is `true` for all `Pn` in `Ps...` where `Pn::polymorphic_query_result_type` is well-formed
 - for all `Pn` in `Ps...` with `Pn::is_requirable == true`, either:
 - `can_require_v<Pn, T>` is `true`, or
 - `can_prefer_v<Pn, T>` is `true`
 - `can_prefer_v<Pn, T>` is `true` for all `Pn` in `Ps...` with `Pn::is_preferable == true`
 - `P::template static_query_v<T> == P::value()` is `true` if `P::template static_query_v<T> == P::value()` is a valid constant expression
 - be valid in the constant expression `is_applicable_property_v<PW, P>`, and that expression shall be `true`.
 - if `P::template static_query_v<PW> == P::value()` is a valid constant expression, then that expression shall be `true`.
- Let `Prop` be a concept-preserving property and let `CP` be a concept-enforcing property. Let `e` be an instance of a type `E` for which `can_require_v<E, Prop>` is `true`. Let `prop` be an instance of type `Prop`. Let `Props...` be a list of properties for which the expression `CP::template polymorphic_wrapper_type<Props...>(e)` is well-formed. If the expression `CP::template static_query_v<E> == CP::value()` is a well-formed constant expression with value `true`, the expression `CP::template polymorphic_wrapper_type<Props...>(std::require(e, prop))` shall be well-formed if and only if, given the type `T = decay_t<decltype(std::require(e, prop))>`,
 - `can_query_v<Pn, T>` is `true` for all `Pn` in `Props...` where `Pn::polymorphic_query_result_type` is well-formed
 - `can_require_concept_v<Pn, T>` is `true` for all `Pn` in `Props...` with `Pn::is_requirable_concept == true`
 - for all `Pn` in `Props...` with `Pn::is_requirable == true`, either:
 - `can_require_v<Pn, T>` is `true`, or
 - `can_prefer_v<Pn, T>` is `true`
 - `can_prefer_v<Pn, T>` is `true` for all `Pn` in `Props...` with `Pn::is_preferable == true`
- [Note: For example, the struct `S` provides the type for a concept-enforcing property:

```
struct S
{
    static constexpr bool is_requirable_concept = true;

    template<class... Ps>
        class polymorphic_wrapper_type;

    using polymorphic_query_result_type = bool;

    template<class T>
        static constexpr bool static_query_v = /* ... */;

    static constexpr bool value() const { return true; }
};
```

—*end note*]